Mass .1

Get A

1. a) Write a JDBC program to insert n student record into STUDENT(rollno,name,percentage) table using Statement.

```
File Name: InsertStudent.java

import java.sql.*;
import java.util.Scanner;

public class InsertStudent {
    public static void main(String args[]) {
        Connection con;
        Statement st;
        Scanner sc = new Scanner(System.in);

        try {
            Class.forName("org.postgresql.Driver");
            con = DriverManager.getConnection("jdbc:postgresql://localhost:5432/testdb", "postgres",
"postgres");
            st = con.createStatement();

            System.out.print("Enter number of students: ");
            int n = sc.nextInt();

            for (int i = 1; i <= n; i++) {
                System.out.print("Enter roll no: ");
                int rno = sc.nextInt();
                sc.nextLine();
                System.out.print("Enter name: ");
                String name = sc.nextLine();
                System.out.print("Enter percentage: ");
                float per = sc.nextFloat();

                String q = "insert into student values(" + rno + ",'" + name + "'," + per + ")";
                st.executeUpdate(q);
            }

            System.out.println("Records inserted successfully");
            con.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

2. b) Write a JDBC program to accept empid and search record from Employee(empid,empname,salary,dept) table using PreparedStatement.

```
File Name: SearchEmployee.java

import java.sql.*;
import java.util.Scanner;

public class SearchEmployee {
    public static void main(String args[]) {
        Connection con;
        PreparedStatement ps;
        ResultSet rs;
        Scanner sc = new Scanner(System.in);

        try {
            Class.forName("org.postgresql.Driver");
            con = DriverManager.getConnection("jdbc:postgresql://localhost:5432/testdb", "postgres",
"postgres");
```

```
            System.out.print("Enter empid: ");
            int id = sc.nextInt();

            ps = con.prepareStatement("select * from employee where empid=?");
            ps.setInt(1, id);
            rs = ps.executeQuery();

            if (rs.next()) {
                System.out.println("Empid: " + rs.getInt(1));
                System.out.println("Empname: " + rs.getString(2));
                System.out.println("Salary: " + rs.getFloat(3));
                System.out.println("Dept: " + rs.getString(4));
            } else {
                System.out.println("Record not found");
            }

            con.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

3. c) Display all records from STUDENT table using Statement and ResultSet.

```
File Name: DisplayStudent.java

import java.sql.*;

public class DisplayStudent {
    public static void main(String args[]) {
        Connection con;
        Statement st;
        ResultSet rs;

        try {
            Class.forName("org.postgresql.Driver");
            con = DriverManager.getConnection("jdbc:postgresql://localhost:5432/testdb", "postgres",
"postgres");
            st = con.createStatement();
            rs = st.executeQuery("select * from student");

            while (rs.next()) {
                System.out.println(rs.getInt(1) + " " + rs.getString(2) + " " + rs.getFloat(3));
            }

            con.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

4. d) PreparedStatement using switch case: Insert, Update and Delete student record.

```
File Name: StudentOperation.java

import java.sql.*;
import java.util.Scanner;

public class StudentOperation {
    public static void main(String args[]) {
        Connection con;
        PreparedStatement ps;
        Scanner sc = new Scanner(System.in);
        int ch;
```

```
        try {
            Class.forName("org.postgresql.Driver");
            con = DriverManager.getConnection("jdbc:postgresql://localhost:5432/testdb", "postgres",
"postgres");

            do {
                System.out.println("1.Insert");
                System.out.println("2.Update");
                System.out.println("3.Delete");
                System.out.println("4.Exit");
                System.out.print("Enter choice: ");
                ch = sc.nextInt();

                switch (ch) {
                    case 1:
                        System.out.print("Enter roll no: ");
                        int r = sc.nextInt();
                        sc.nextLine();
                        System.out.print("Enter name: ");
                        String n = sc.nextLine();
                        System.out.print("Enter percentage: ");
                        float p = sc.nextFloat();

                        ps = con.prepareStatement("insert into student values(?,?,?)");
                        ps.setInt(1, r);
                        ps.setString(2, n);
                        ps.setFloat(3, p);
                        ps.executeUpdate();
                        System.out.println("Record inserted");
                        break;

                    case 2:
                        System.out.print("Enter roll no: ");
                        int r1 = sc.nextInt();
                        System.out.print("Enter new percentage: ");
                        float p1 = sc.nextFloat();

                        ps = con.prepareStatement("update student set percentage=? where rollno=?");
                        ps.setFloat(1, p1);
                        ps.setInt(2, r1);
                        ps.executeUpdate();
                        System.out.println("Record updated");
                        break;

                    case 3:
                        System.out.print("Enter roll no: ");
                        int r2 = sc.nextInt();

                        ps = con.prepareStatement("delete from student where rollno=?");
                        ps.setInt(1, r2);
                        ps.executeUpdate();
                        System.out.println("Record deleted");
                        break;

                    case 4:
                        System.out.println("Exit");
                        break;
                }
            } while (ch != 4);

            con.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

5. e) Program using DatabaseMetaData to display database product name, driver name and list of tables.

```
File Name: DBMetaDataDemo.java

import java.sql.*;

public class DBMetaDataDemo {
    public static void main(String args[]) {
        Connection con;
        DatabaseMetaData dbmd;
        ResultSet rs;

        try {
            Class.forName("org.postgresql.Driver");
            con = DriverManager.getConnection("jdbc:postgresql://localhost:5432/testdb", "postgres",
"postgres");

            dbmd = con.getMetaData();

            System.out.println("Database Product Name: " + dbmd.getDatabaseProductName());
            System.out.println("Driver Name: " + dbmd.getDriverName());

            rs = dbmd.getTables(null, null, "%", new String[]{"TABLE"});
            System.out.println("Tables are:");
            while (rs.next()) {
                System.out.println(rs.getString("TABLE_NAME"));
            }

            con.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

6. f) Program using ResultSetMetaData to print column names, data types and table name of STUDENT table.

```
File Name: RSMDemo.java

import java.sql.*;

public class RSMDemo {
    public static void main(String args[]) {
        Connection con;
        Statement st;
        ResultSet rs;
        ResultSetMetaData rsmd;

        try {
            Class.forName("org.postgresql.Driver");
            con = DriverManager.getConnection("jdbc:postgresql://localhost:5432/testdb", "postgres",
"postgres");
            st = con.createStatement();

            rs = st.executeQuery("select * from student");
            rsmd = rs.getMetaData();

            int n = rsmd.getColumnCount();

            for (int i = 1; i <= n; i++) {
                System.out.println("Column Name: " + rsmd.getColumnName(i));
                System.out.println("Data Type: " + rsmd.getColumnTypeName(i));
                System.out.println("Table Name: " + rsmd.getTableName(i));
            }

            con.close();
        } catch (Exception e) {
            System.out.println(e);
        }
```

```
    }
}
```

Get B

1. a) Menu driven program on STUDENT using PreparedStatement and switch case.

```
File Name: StudentMenu.java

import java.sql.*;
import java.util.Scanner;

public class StudentMenu {
    public static void main(String args[]) {
        Connection con;
        PreparedStatement ps;
        ResultSet rs;
        Scanner sc = new Scanner(System.in);
        int ch;

        try {
            Class.forName("org.postgresql.Driver");
            con = DriverManager.getConnection("jdbc:postgresql://localhost:5432/testdb", "postgres",
"postgres");

            do {
                System.out.println("1.Insert");
                System.out.println("2.Modify");
                System.out.println("3.Delete");
                System.out.println("4.Search");
                System.out.println("5.View All");
                System.out.println("6.Exit");
                System.out.print("Enter choice: ");
                ch = sc.nextInt();

                switch (ch) {
                    case 1:
                        System.out.print("Enter rollno: ");
                        int r = sc.nextInt();
                        sc.nextLine();
                        System.out.print("Enter name: ");
                        String n = sc.nextLine();
                        System.out.print("Enter class: ");
                        String c = sc.nextLine();
                        System.out.print("Enter total marks: ");
                        int m = sc.nextInt();

                        ps = con.prepareStatement("insert into student values(?,?,?,?)");
                        ps.setInt(1, r);
                        ps.setString(2, n);
                        ps.setString(3, c);
                        ps.setInt(4, m);
                        ps.executeUpdate();
                        System.out.println("Record inserted");
                        break;

                    case 2:
                        System.out.print("Enter rollno to modify: ");
                        int r1 = sc.nextInt();
                        sc.nextLine();
                        System.out.print("Enter new name: ");
                        String n1 = sc.nextLine();
                        System.out.print("Enter new class: ");
                        String c1 = sc.nextLine();
                        System.out.print("Enter new total marks: ");
                        int m1 = sc.nextInt();
```

```
                        ps = con.prepareStatement("update student set name=?, class=?, totalmarks=?
                       where rollno=?");
                        ps.setString(1, n1);
                        ps.setString(2, c1);
                        ps.setInt(3, m1);
                        ps.setInt(4, r1);
                        ps.executeUpdate();
                        System.out.println("Record modified");
                        break;

                    case 3:
                        System.out.print("Enter rollno to delete: ");
                        int r2 = sc.nextInt();
                        ps = con.prepareStatement("delete from student where rollno=?");
                        ps.setInt(1, r2);
                        ps.executeUpdate();
                        System.out.println("Record deleted");
                        break;

                    case 4:
                        System.out.print("Enter rollno to search: ");
                        int r3 = sc.nextInt();
                        ps = con.prepareStatement("select * from student where rollno=?");
                        ps.setInt(1, r3);
                        rs = ps.executeQuery();

                        if (rs.next()) {
                            System.out.println("Rollno: " + rs.getInt(1));
                            System.out.println("Name: " + rs.getString(2));
                            System.out.println("Class: " + rs.getString(3));
                            System.out.println("Total Marks: " + rs.getInt(4));
                        } else {
                            System.out.println("Record not found");
                        }
                        break;

                    case 5:
                        ps = con.prepareStatement("select * from student");
                        rs = ps.executeQuery();
                        while (rs.next()) {
                            System.out.println(rs.getInt(1) + " " + rs.getString(2) + " " +
                                                rs.getString(3) + " " + rs.getInt(4));
                        }
                        break;

                    case 6:
                        System.out.println("Exit");
                        break;
                }
            } while (ch != 6);

            con.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

2. b) Menu driven program on MOBILE table using switch case.

```
File Name: MobileMenu.java

import java.sql.*;
import java.util.Scanner;

public class MobileMenu {
    public static void main(String args[]) {
        Connection con;
        PreparedStatement ps;
        ResultSet rs;
        Scanner sc = new Scanner(System.in);
        int ch;
```

```
        try {
            Class.forName("org.postgresql.Driver");
            con = DriverManager.getConnection("jdbc:postgresql://localhost:5432/testdb", "postgres",
"postgres");

            do {
                System.out.println("1.Insert");
                System.out.println("2.Modify");
                System.out.println("3.Delete");
                System.out.println("4.Search");
                System.out.println("5.View All");
                System.out.println("6.Exit");
                System.out.print("Enter choice: ");
                ch = sc.nextInt();
                sc.nextLine();

                switch (ch) {
                    case 1:
                        System.out.print("Enter Model Number: ");
                        String mno = sc.nextLine();
                        System.out.print("Enter Model Name: ");
                        String mname = sc.nextLine();
                        System.out.print("Enter Model Color: ");
                        String color = sc.nextLine();
                        System.out.print("Enter Sim Type: ");
                        String sim = sc.nextLine();
                        System.out.print("Enter Network Type: ");
                        String net = sc.nextLine();
                        System.out.print("Enter Battery Capacity: ");
                        int battery = sc.nextInt();
                        System.out.print("Enter Internal Storage: ");
                        int storage = sc.nextInt();
                        System.out.print("Enter RAM: ");
                        int ram = sc.nextInt();
                        sc.nextLine();
                        System.out.print("Enter Processor Type: ");
                        String processor = sc.nextLine();

                        ps = con.prepareStatement("insert into mobile values(?,?,?,?,?,?,?,?,?)");
                        ps.setString(1, mno);
                        ps.setString(2, mname);
                        ps.setString(3, color);
                        ps.setString(4, sim);
                        ps.setString(5, net);
                        ps.setInt(6, battery);
                        ps.setInt(7, storage);
                        ps.setInt(8, ram);
                        ps.setString(9, processor);
                        ps.executeUpdate();
                        System.out.println("Record inserted");
                        break;

                    case 2:
                        System.out.print("Enter Model Number to modify: ");
                        String mno1 = sc.nextLine();
                        System.out.print("Enter New Model Name: ");
                        String mname1 = sc.nextLine();
                        System.out.print("Enter New Model Color: ");
                        String color1 = sc.nextLine();
                        System.out.print("Enter New Sim Type: ");
                        String sim1 = sc.nextLine();
                        System.out.print("Enter New Network Type: ");
                        String net1 = sc.nextLine();
                        System.out.print("Enter New Battery Capacity: ");
                        int battery1 = sc.nextInt();
                        System.out.print("Enter New Internal Storage: ");
                        int storage1 = sc.nextInt();
                        System.out.print("Enter New RAM: ");
                        int ram1 = sc.nextInt();
                        sc.nextLine();
                        System.out.print("Enter New Processor Type: ");
                        String processor1 = sc.nextLine();
```

```
                        ps = con.prepareStatement("update mobile set model_name=?, model_color=?,
sim_type=?, networktype=?, batterycapacity=?, internalstorage=?, ram=?, processortype=? where
model_number=?");
                        ps.setString(1, mname1);
                        ps.setString(2, color1);
                        ps.setString(3, sim1);
                        ps.setString(4, net1);
                        ps.setInt(5, battery1);
                        ps.setInt(6, storage1);
                        ps.setInt(7, ram1);
                        ps.setString(8, processor1);
                        ps.setString(9, mno1);
                        ps.executeUpdate();
                        System.out.println("Record modified");
                        break;

                    case 3:
                        System.out.print("Enter Model Number to delete: ");
                        String mno2 = sc.nextLine();
                        ps = con.prepareStatement("delete from mobile where model_number=?");
                        ps.setString(1, mno2);
                        ps.executeUpdate();
                        System.out.println("Record deleted");
                        break;

                    case 4:
                        System.out.print("Enter Model Number to search: ");
                        String mno3 = sc.nextLine();
                        ps = con.prepareStatement("select * from mobile where model_number=?");
                        ps.setString(1, mno3);
                        rs = ps.executeQuery();

                        if (rs.next()) {
                            System.out.println(rs.getString(1) + " " + rs.getString(2) + " " +
rs.getString(3));
                        } else {
                            System.out.println("Record not found");
                        }
                        break;

                    case 5:
                        ps = con.prepareStatement("select * from mobile");
                        rs = ps.executeQuery();
                        while (rs.next()) {
                            System.out.println(rs.getString(1) + " " + rs.getString(2) + " " +
rs.getString(3));
                        }
                        break;

                    case 6:
                        System.out.println("Exit");
                        break;
                }
            } while (ch != 6);

            con.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

3. c) Program using scrollable ResultSet on Employee table.

```
File Name: ScrollEmployee.java

import java.sql.*;

public class ScrollEmployee {
    public static void main(String args[]) {
        Connection con;
        Statement st;
        ResultSet rs;
```

```
        try {
            Class.forName("org.postgresql.Driver");
            con = DriverManager.getConnection("jdbc:postgresql://localhost:5432/testdb", "postgres",
"postgres");

            st = con.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_READ_ONLY);
            rs = st.executeQuery("select * from employee");

            rs.first();
            System.out.println("First Record: " + rs.getInt(1) + " " + rs.getString(2));

            rs.last();
            System.out.println("Last Record: " + rs.getInt(1) + " " + rs.getString(2));

            rs.absolute(2);
            System.out.println("Second Record: " + rs.getInt(1) + " " + rs.getString(2));

            rs.previous();
            System.out.println("Previous Record: " + rs.getInt(1) + " " + rs.getString(2));

            rs.next();
            System.out.println("Next Record: " + rs.getInt(1) + " " + rs.getString(2));

            con.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

4. d) Program using updatable ResultSet to update and delete row from STUDENT table.

```
File Name: UpdateDeleteResultSet.java

import java.sql.*;

public class UpdateDeleteResultSet {
    public static void main(String args[]) {
        Connection con;
        Statement st;
        ResultSet rs;

        try {
            Class.forName("org.postgresql.Driver");
            con = DriverManager.getConnection("jdbc:postgresql://localhost:5432/testdb", "postgres",
"postgres");

            st = con.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE, ResultSet.CONCUR_UPDATABLE);
            rs = st.executeQuery("select * from student");

            while (rs.next()) {
                if (rs.getInt("rollno") == 1) {
                    rs.updateString("name", "Rohan");
                    rs.updateInt("totalmarks", 480);
                    rs.updateRow();
                }
            }

            rs.beforeFirst();
            while (rs.next()) {
                if (rs.getInt("rollno") == 2) {
                    rs.deleteRow();
                }
            }

            con.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Get C

1. a) Program to demonstrate transaction using commit() and rollback() on STUDENT table.

```
File Name: TransactionDemo.java

import java.sql.*;

public class TransactionDemo {
    public static void main(String args[]) {
        Connection con = null;
        Statement st = null;

        try {
            Class.forName("org.postgresql.Driver");
            con = DriverManager.getConnection("jdbc:postgresql://localhost:5432/testdb", "postgres",
"postgres");

            con.setAutoCommit(false);
            st = con.createStatement();

            st.executeUpdate("insert into student values(1,'Amit',78.5)");
            st.executeUpdate("insert into student values(2,'Neha',92)");

            if (92 == 92) {
                con.rollback();
                System.out.println("Transaction rollback.");
            } else {
                con.commit();
                System.out.println("Transaction committed.");
            }

            con.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

2. b) Program to use Savepoint on Employee table.

```
File Name: SavepointDemo.java

import java.sql.*;

public class SavepointDemo {
    public static void main(String args[]) {
        Connection con = null;
        Statement st = null;
        Savepoint sp;

        try {
            Class.forName("org.postgresql.Driver");
            con = DriverManager.getConnection("jdbc:postgresql://localhost:5432/testdb", "postgres",
"postgres");

            con.setAutoCommit(false);
            st = con.createStatement();

            st.executeUpdate("insert into employee values(104,'Ravi',40000,'HR')");
            sp = con.setSavepoint("sp1");

            st.executeUpdate("update employee set salary=50000 where empid=104");

            con.rollback(sp);
            con.commit();

            System.out.println("Rollback to savepoint and commit done.");
```

```
            con.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Mass . 2

Get A

1. a) Servlet to display current system date and time.

```
File Name: DateTimeServlet.java

import java.io.*;
import java.util.Date;
import javax.servlet.*;
import javax.servlet.http.*;

public class DateTimeServlet extends HttpServlet {
    public void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException,
ServletException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        out.println("<html><body>");
        out.println("<h2>Current Date and Time: " + new Date() + "</h2>");
        out.println("</body></html>");
    }
}
```

2. b) Servlet to display client request info and server info.

```
File Name: InfoServlet.java

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class InfoServlet extends HttpServlet {
    public void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException,
ServletException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        out.println("IP Address: " + req.getRemoteAddr() + "<br>");
        out.println("Browser Type: " + req.getHeader("User-Agent") + "<br>");
        out.println("Operating System: " + System.getProperty("os.name") + "<br>");
        out.println("Server Name: " + req.getServerName() + "<br>");
    }
}
```

3. c) JSP implicit objects program.

```
File Name: implicit.jsp

<%@ page import="java.util.Date" %>
<html>
<body>
<%
out.println("Current Date and Time: " + new Date());
out.println("<br>Header Information: " + request.getHeader("User-Agent"));
response.getWriter().println("<br>Message using response object");
out.println("<br>Session ID: " + session.getId());
Cookie c = new Cookie("uname", "Amit");
response.addCookie(c);
out.println("<br>Cookie Added Successfully");
%>
</body>
</html>
```

4. d) Login page using HTML and Servlet.

```
File Name: login.html

<html>
<body>
<form action="LoginServlet" method="post">
    Username: <input type="text" name="uname"><br><br>
    Password: <input type="password" name="pass"><br><br>
    <input type="submit" value="Login">
    <input type="reset" value="Clear">
</form>
</body>
</html>

File Name: LoginServlet.java

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class LoginServlet extends HttpServlet {
    public void doPost(HttpServletRequest req, HttpServletResponse res) throws IOException,
ServletException {
        PrintWriter out = res.getWriter();
        String uname = req.getParameter("uname");
        String pass = req.getParameter("pass");

        if (uname.equals("admin") && pass.equals("1234"))
            out.println("<h2>Login Successful</h2>");
        else
            out.println("<h2>Invalid Username or Password</h2>");
    }
}
```

5. e) Addition using servlet POST method.

```
File Name: add.html

<html>
<body>
<form action="AddServlet" method="post">
    Enter First Number: <input type="text" name="n1"><br><br>
    Enter Second Number: <input type="text" name="n2"><br><br>
    <input type="submit" value="Add">
</form>
</body>
</html>
```

```
File Name: AddServlet.java
-----------------------------------------------------------------------------------------------
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class AddServlet extends HttpServlet {
    public void doPost(HttpServletRequest req, HttpServletResponse res) throws IOException,
ServletException {
        PrintWriter out = res.getWriter();
        int a = Integer.parseInt(req.getParameter("n1"));
        int b = Integer.parseInt(req.getParameter("n2"));
        out.println("<h2>Addition = " + (a + b) + "</h2>");
    }
}
```

6. f) Greeting message based on server time.

```
File Name: GreetingServlet.java

import java.io.*;
import java.time.LocalTime;
import javax.servlet.*;
import javax.servlet.http.*;

public class GreetingServlet extends HttpServlet {
    public void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException,
ServletException {
        PrintWriter out = res.getWriter();
        int hour = LocalTime.now().getHour();
        String msg;

        if (hour >= 5 && hour < 12)
            msg = "Good Morning";
        else if (hour >= 12 && hour < 17)
            msg = "Good Afternoon";
        else if (hour >= 17 && hour < 21)
            msg = "Good Evening";
        else
            msg = "Good Night";

        out.println("<h2>" + msg + "</h2>");
    }
}
```

7. g) Online Book Purchase using HttpSession.

```
File Name: BookLoginServlet.java

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class BookLoginServlet extends HttpServlet {
    public void doPost(HttpServletRequest req, HttpServletResponse res) throws IOException,
ServletException {
        String uname = req.getParameter("uname");
        String pass = req.getParameter("pass");

        if (uname.equals("admin") && pass.equals("123")) {
            HttpSession session = req.getSession();
            session.setAttribute("user", uname);
            res.sendRedirect("book.html");
        } else {
            res.getWriter().println("Invalid Login");
        }
    }
}

-----------------------------------------------------------------------------------------------\
```

```
File Name: BookServlet.java

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class BookServlet extends HttpServlet {
    public void doPost(HttpServletRequest req, HttpServletResponse res) throws IOException,
ServletException {
        HttpSession session = req.getSession(false);
        PrintWriter out = res.getWriter();

        if (session == null) {
            out.println("Please login first");
            return;
        }

        String bname = req.getParameter("bname");
        int price = Integer.parseInt(req.getParameter("price"));
        int qty = Integer.parseInt(req.getParameter("qty"));

        int total = price * qty;
        out.println("<h2>Book Name: " + bname + "</h2>");
        out.println("<h2>Total Bill: " + total + "</h2>");
    }
}
```

Get B

1. a) Customer search servlet using HTML page and database.

```
File Name: search.html

<html>
<body>
<form action="SearchCustomerServlet" method="post">
    Enter Customer Number: <input type="text" name="cno"><br><br>
    <input type="submit" value="Search">
</form>
</body>
</html>


-----------------------------------------------------------------------------------------------------


File Name: SearchCustomerServlet.java

import java.io.*;
import java.sql.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class SearchCustomerServlet extends HttpServlet {
    public void doPost(HttpServletRequest req, HttpServletResponse res) throws IOException,
ServletException {
        PrintWriter out = res.getWriter();
        String cno = req.getParameter("cno");

        try {
            Class.forName("org.postgresql.Driver");
            Connection con =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/testdb","postgres","postgres");
            PreparedStatement ps = con.prepareStatement("select * from customer where customerno=?");
            ps.setInt(1, Integer.parseInt(cno));
            ResultSet rs = ps.executeQuery();
```

```
            if (rs.next()) {
                out.println("Customer No: " + rs.getInt(1) + "<br>");
                out.println("Customer Name: " + rs.getString(2) + "<br>");
                out.println("Customer Address: " + rs.getString(3) + "<br>");
            } else {
                out.println("Customer not found");
            }

            con.close();
        } catch (Exception e) {
            out.println(e);
        }
    }
}
```

2. b) Registration form using servlet.

```
File Name: register.html

<html>
<body>
<form action="RegisterServlet" method="post">
    Name: <input type="text" name="name"><br><br>
    Email: <input type="text" name="email"><br><br>
    Gender:
    <input type="radio" name="gender" value="Male">Male
    <input type="radio" name="gender" value="Female">Female<br><br>
    Course:
    <select name="course">
        <option>BSc</option>
        <option>BCA</option>
        <option>MSc</option>
    </select><br><br>
    <input type="submit" value="Register">
</form>
</body>
</html>

File Name: RegisterServlet.java

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class RegisterServlet extends HttpServlet {
    public void doPost(HttpServletRequest req, HttpServletResponse res) throws IOException,
ServletException {
        PrintWriter out = res.getWriter();
        out.println("<h3>Registration Confirmation</h3>");
        out.println("Name: " + req.getParameter("name") + "<br>");
        out.println("Email: " + req.getParameter("email") + "<br>");
        out.println("Gender: " + req.getParameter("gender") + "<br>");
        out.println("Course: " + req.getParameter("course") + "<br>");
    }
}
```

3. c) Hit counter using HttpSession.

```
File Name: HitCountServlet.java

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class HitCountServlet extends HttpServlet {
    public void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException,
ServletException {
        PrintWriter out = res.getWriter();
        HttpSession session = req.getSession();
        Integer count = (Integer) session.getAttribute("hit");
```

```
        if (count == null)
            count = 1;
        else
            count = count + 1;

        session.setAttribute("hit", count);
        out.println("<h3>Page opened " + count + " times</h3>");
    }
}
```

4. d) Subject cookie program with no duplicate cookies.

```
File Name: subject.html


<html>
<body>
<form action="SubjectServlet" method="post">
    <input type="radio" name="subject" value="Maths">Maths
    <input type="radio" name="subject" value="Physics">Physics
    <input type="radio" name="subject" value="Chemistry">Chemistry
    <input type="radio" name="subject" value="Biology">Biology
    <br><br>
    <input type="submit" value="Submit">
</form>
</body>
</html>
```

```
File Name: SubjectServlet.java

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class SubjectServlet extends HttpServlet {
    public void doPost(HttpServletRequest req, HttpServletResponse res) throws IOException,
ServletException {
        PrintWriter out = res.getWriter();
        String subject = req.getParameter("subject");
        boolean found = false;

        Cookie ck[] = req.getCookies();
        if (ck != null) {
            for (Cookie c : ck) {
                if (c.getName().equals("subject") && c.getValue().equals(subject)) {
                    found = true;
                    break;
                }
            }
        }
        if (!found) {
            Cookie c1 = new Cookie("subject", subject);
            res.addCookie(c1);
            out.println("Cookie added for subject: " + subject);
        } else {
            out.println("Duplicate cookie not allowed");
        }
    }
}
```

5. e) Login page using JSP.

```
File Name: login.jsp

<html>
<body>
<form method="post">
    Username: <input type="text" name="uname"><br><br>
    Password: <input type="password" name="pass"><br><br>
    <input type="submit" value="Login">
</form>

<%
String u = request.getParameter("uname");
String p = request.getParameter("pass");

if(u != null && p != null)
{
    if(u.equals("admin") && p.equals("123"))
        out.println("<h3>Login Successful</h3>");
    else
        out.println("<h3>Invalid Username or Password</h3>");
}
%>
</body>
</html>
```

6. f) Display PATIENT table details in JSP tabular form.

```
File Name: patient.jsp

<%@ page import="java.sql.*" %>
<html>
<body>
<table border="1">
<tr>
    <th>PatientNo</th>
    <th>PatientName</th>
    <th>PatientAddress</th>
    <th>PatientAge</th>
    <th>PatientDisease</th>
</tr>

<%
try {
    Class.forName("org.postgresql.Driver");
    Connection con =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/testdb","postgres","postgres");
    Statement st = con.createStatement();
    ResultSet rs = st.executeQuery("select * from patient");

    while(rs.next()) {
%>
<tr>
    <td><%= rs.getInt(1) %></td>
    <td><%= rs.getString(2) %></td>
    <td><%= rs.getString(3) %></td>
    <td><%= rs.getInt(4) %></td>
    <td><%= rs.getString(5) %></td>
</tr>
<%
    }
    con.close();
} catch(Exception e) {
    out.println(e);
}
%>
</table>
</body>
</html>
```

7. g) JSP program to add two numbers using POST method.

```
File Name: add.jsp

<html>
<body>
<form action="add.jsp" method="post">
    Enter First Number: <input type="text" name="n1"><br><br>
    Enter Second Number: <input type="text" name="n2"><br><br>
    <input type="submit" value="Add">
</form>

<%
String a = request.getParameter("n1");
String b = request.getParameter("n2");

if(a != null && b != null) {
    int n1 = Integer.parseInt(a);
    int n2 = Integer.parseInt(b);
    int sum = n1 + n2;
    out.println("<h3>Addition = " + sum + "</h3>");
}
%>
</body>
</html>
```

Mass . 3

Get A

1. a) Create a Spring Boot application to display Hello World.

```
Save in Demo project:

Main File:
demo/src/main/java/com/example/demo/DemoApplication.java

Controller File:
demo/src/main/java/com/example/demo/HelloController.java

Program:
-----------------------------------------------------------------------------------------------
// DemoApplication.java
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}


-----------------------------------------------------------------------------------------------
// HelloController.java
package com.example.demo;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {
    @GetMapping("/hello")
    public String hello() {
        return "Hello World";
    }
}
```

2. b) Create a Spring Boot application to display current system date and time.

```
Save in Demo project:

Main File:
demo/src/main/java/com/example/demo/DemoApplication.java

Controller File:
demo/src/main/java/com/example/demo/DateTimeController.java

Program:


-----------------------------------------------------------------------------------------------

// DateTimeController.java
package com.example.demo;

import java.time.LocalDateTime;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class DateTimeController {
    @GetMapping("/datetime")
    public String getDateTime() {
        return "Current Date and Time: " + LocalDateTime.now();
```

```
    }
}
```

3. c) Develop a Spring Boot application using controller to perform addition, subtraction, division, multiplication.

```
Save in Demo project:

Main File:
demo/src/main/java/com/example/demo/DemoApplication.java

Controller File:
demo/src/main/java/com/example/demo/CalculatorController.java

Program:

-----------------------------------------------------------------------------------------------------

// CalculatorController.java
package com.example.demo;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class CalculatorController {

    @GetMapping("/add")
    public String add(@RequestParam int a, @RequestParam int b) {
        return "Addition = " + (a + b);
    }

    @GetMapping("/sub")
    public String sub(@RequestParam int a, @RequestParam int b) {
        return "Subtraction = " + (a - b);
    }

    @GetMapping("/mul")
    public String mul(@RequestParam int a, @RequestParam int b) {
        return "Multiplication = " + (a * b);
    }

    @GetMapping("/div")
    public String div(@RequestParam int a, @RequestParam int b) {
        if (b == 0) {
            return "Division by zero not allowed";
        }
        return "Division = " + (a / b);
    }
}
```

4. d) Design a simple login application using Spring Boot REST controller.

```
Save in Demo project:

Main File:
demo/src/main/java/com/example/demo/DemoApplication.java

Controller File:
demo/src/main/java/com/example/demo/LoginController.java

Program:



-----------------------------------------------------------------------------------------------------
// LoginController.java
package com.example.demo;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController
public class LoginController {

    @GetMapping("/login")
    public String login(@RequestParam String username, @RequestParam String password) {
        if (username.equals("admin") && password.equals("1234")) {
            return "Login Successful";
        } else {
            return "Invalid Username or Password";
        }
    }
}
```

5. e) String operations API: replace, reverse, word count, palindrome, case conversion.

```
Save in Demo project:

Main File:
demo/src/main/java/com/example/demo/DemoApplication.java

Controller File:
demo/src/main/java/com/example/demo/StringController.java

Program:

-----------------------------------------------------------------------------------------------------

// StringController.java
package com.example.demo;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class StringController {

    @GetMapping("/replace")
    public String replaceText(@RequestParam String text,
                              @RequestParam String target,
                              @RequestParam String replacement) {
        return text.replace(target, replacement);
    }

    @GetMapping("/reverse")
    public String reverseText(@RequestParam String text) {
        String rev = "";
        for (int i = text.length() - 1; i >= 0; i--) {
            rev = rev + text.charAt(i);
        }
        return rev;
    }

    @GetMapping("/count")
    public String countWords(@RequestParam String text) {
        String arr[] = text.trim().split("\\s+");
        return "Word Count = " + arr.length;
    }

    @GetMapping("/check-palindrome")
    public String checkPalindrome(@RequestParam String text) {
        String rev = "";
        for (int i = text.length() - 1; i >= 0; i--) {
            rev = rev + text.charAt(i);
        }
        if (text.equals(rev))
            return "true";
        else
            return "false";
    }
```

```
    @GetMapping("/convert-case")
    public String convertCase(@RequestParam String text, @RequestParam String type) {
        if (type.equalsIgnoreCase("upper")) {
            return text.toUpperCase();
        } else if (type.equalsIgnoreCase("lower")) {
            return text.toLowerCase();
        } else {
            return "Invalid type";
        }
    }
}
```

Get B

1. a) Registration form using Spring Boot.

```
Save in Demo project:

Main File:
demo/src/main/java/com/example/demo/DemoApplication.java

Controller File:
demo/src/main/java/com/example/demo/StudentController.java

HTML Files:
demo/src/main/resources/templates/studentform.html
demo/src/main/resources/templates/result.html

Program:

-----------------------------------------------------------------------------------------------

// StudentController.java
package com.example.demo;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;

@Controller
public class StudentController {

    @GetMapping("/form")
    public String showForm() {
        return "studentform";
    }

    @PostMapping("/register")
    public String registerStudent(@RequestParam String name,
                                  @RequestParam String email,
                                  @RequestParam String gender,
                                  @RequestParam String course,
                                  Model model) {
        model.addAttribute("name", name);
        model.addAttribute("email", email);
        model.addAttribute("gender", gender);
        model.addAttribute("course", course);
        return "result";
    }
}


-----------------------------------------------------------------------------------------------
<!-- studentform.html -->
<!DOCTYPE html>
<html>
<body>
    <h2>Student Registration Form</h2>
    <form action="/register" method="post">
        Name: <input type="text" name="name"><br><br>
        Email: <input type="text" name="email"><br><br>
```

```
        Gender:
        <input type="radio" name="gender" value="Male">Male
        <input type="radio" name="gender" value="Female">Female
        <br><br>
        Course:
        <select name="course">
            <option value="BSc">BSc</option>
            <option value="BCA">BCA</option>
            <option value="MSc">MSc</option>
        </select>
        <br><br>
        <input type="submit" value="Submit">
    </form>
</body>
</html>

-----------------------------------------------------------------------------------------------------

<!-- result.html -->
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<body>
    <h2>Student Details</h2>
    Name: <span th:text="${name}"></span><br><br>
    Email: <span th:text="${email}"></span><br><br>
    Gender: <span th:text="${gender}"></span><br><br>
    Course: <span th:text="${course}"></span>
</body>
</html>
```

2. b) Form validation using Product(Name, Price, Quantity).

```
Save in Demo project:

Main File:
demo/src/main/java/com/example/demo/DemoApplication.java

Model File:
demo/src/main/java/com/example/demo/Product.java

Controller File:
demo/src/main/java/com/example/demo/ProductController.java

HTML Files:
demo/src/main/resources/templates/productform.html
demo/src/main/resources/templates/productresult.html

Program:
-----------------------------------------------------------------------------------------------------

// Product.java
package com.example.demo;

import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotEmpty;

public class Product {

    @NotEmpty(message = "Product name should not be empty")
    private String name;

    @Min(value = 1, message = "Price must be positive")
    private int price;

    @Min(value = 1, message = "Quantity must be positive")
    private int quantity;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
```

```
    public int getPrice() {
        return price;
    }

    public void setPrice(int price) {
        this.price = price;
    }

    public int getQuantity() {
        return quantity;
    }

    public void setQuantity(int quantity) {
        this.quantity = quantity;
    }
}
```

```
// ProductController.java
package com.example.demo;

import jakarta.validation.Valid;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;

@Controller
public class ProductController {

    @GetMapping("/product")
    public String showForm(Product product) {
        return "productform";
    }

    @PostMapping("/saveProduct")
    public String saveProduct(@Valid Product product, BindingResult result, Model model) {
        if (result.hasErrors()) {
            return "productform";
        }
        model.addAttribute("product", product);
        return "productresult";
    }
}
```

```
<!-- productform.html -->
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<body>
    <h2>Product Form</h2>
    <form action="/saveProduct" method="post" th:object="${product}">
        Name: <input type="text" th:field="*{name}"><br>
        <span th:errors="*{name}"></span><br><br>

        Price: <input type="text" th:field="*{price}"><br>
        <span th:errors="*{price}"></span><br><br>

        Quantity: <input type="text" th:field="*{quantity}"><br>
        <span th:errors="*{quantity}"></span><br><br>

        <input type="submit" value="Submit">
    </form>
</body>
</html>
```

```
<!-- productresult.html -->
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<body>
    <h2>Product Details</h2>
    Name: <span th:text="${product.name}"></span><br><br>
    Price: <span th:text="${product.price}"></span><br><br>
    Quantity: <span th:text="${product.quantity}"></span>
```

```
</body>
</html>
```

3. c) PathVariable and RequestParam program.

```
Save in Demo project:

Main File:
demo/src/main/java/com/example/demo/DemoApplication.java

Controller File:
demo/src/main/java/com/example/demo/MathController.java

Program:
-----------------------------------------------------------------------------------------------

// MathController.java
package com.example.demo;

import org.springframework.web.bind.annotation.*;

@RestController
public class MathController {

    @GetMapping("/multiply/{a}/{b}")
    public String multiply(@PathVariable int a, @PathVariable int b) {
        return "Multiplication = " + (a * b);
    }

    @GetMapping("/add2")
    public String add(@RequestParam int a, @RequestParam int b) {
        return "Addition = " + (a + b);
    }
}
```

4. d) String replacement REST API with exception handling.

```
Save in Demo project:

Main File:
demo/src/main/java/com/example/demo/DemoApplication.java

Controller File:
demo/src/main/java/com/example/demo/StringReplaceController.java

Exception File:
demo/src/main/java/com/example/demo/CustomException.java

Exception Handler File:
demo/src/main/java/com/example/demo/GlobalExceptionHandler.java

Program:
-----------------------------------------------------------------------------------------------

// StringReplaceController.java
package com.example.demo;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class StringReplaceController {

    @GetMapping("/replace2")
    public String replaceText(@RequestParam String text,
                              @RequestParam String target,
                              @RequestParam String replacement) {
        if (text == null || text.isEmpty() || target == null || target.isEmpty()) {
            throw new CustomException("Input string or target cannot be empty");
        }
        return text.replace(target, replacement);
    }
```

```
}


// CustomException.java
package com.example.demo;

public class CustomException extends RuntimeException {
    public CustomException(String message) {
        super(message);
    }
}
-----------------------------------------------------------------------------------------------------

// GlobalExceptionHandler.java
package com.example.demo;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(CustomException.class)
    public ResponseEntity<String> handleCustomException(CustomException ex) {
        return new ResponseEntity<>(ex.getMessage(), HttpStatus.BAD_REQUEST);
    }
}
```

Mass . 4

Get A

1. 1) Exception Handling: Search User by ID with 404 custom message.

```
Save in Demo project:

Main File:
demo/src/main/java/com/example/demo/DemoApplication.java

Controller File:
demo/src/main/java/com/example/demo/UserController.java

Exception File:
demo/src/main/java/com/example/demo/UserNotFoundException.java

Exception Handler File:
demo/src/main/java/com/example/demo/GlobalUserExceptionHandler.java

Program:

-----------------------------------------------------------------------------------------------------
// UserController.java
package com.example.demo;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class UserController {

    @GetMapping("/users/{id}")
    public String getUserById(@PathVariable int id) {
        if (id == 1) {
            return "User found: Amit";
        } else if (id == 2) {
            return "User found: Neha";
        } else {
            throw new UserNotFoundException("User with ID " + id + " not found");
```

```
        }
    }
}

// UserNotFoundException.java
package com.example.demo;

public class UserNotFoundException extends RuntimeException {
    public UserNotFoundException(String message) {
        super(message);
    }
}
-----------------------------------------------------------------------------------------------------

// GlobalUserExceptionHandler.java
package com.example.demo;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ExceptionHandler;

@ControllerAdvice
public class GlobalUserExceptionHandler {

    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<String> handleUserNotFound(UserNotFoundException ex) {
        return new ResponseEntity<>(ex.getMessage(), HttpStatus.NOT_FOUND);
    }
}
```

2. 2) Resource-Based URL Design: Library System API.

```
Save in Demo project:

Main File:
demo/src/main/java/com/example/demo/DemoApplication.java

Controller File:
demo/src/main/java/com/example/demo/BookController.java

Program:
-----------------------------------------------------------------------------------------------------

// BookController.java
package com.example.demo;

import java.util.Arrays;
import java.util.List;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/books")
public class BookController {

    @GetMapping
    public List<String> getBooks() {
        return Arrays.asList("Java", "Spring Boot", "Python");
    }

    @PostMapping
    public String addBook(@RequestBody String title) {
        return "Book " + title + " Added";
    }

    @DeleteMapping("/{id}")
    public String deleteBook(@PathVariable int id) {
        return "Book with ID " + id + " deleted";
    }
}
```

3. 3) Search & Filter Logic: Product Search API.

```
Save in Demo project:

Main File:
demo/src/main/java/com/example/demo/DemoApplication.java

Controller File:
demo/src/main/java/com/example/demo/ProductSearchController.java

Program:


-----------------------------------------------------------------------------------------------
// ProductSearchController.java
package com.example.demo;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class ProductSearchController {

    @GetMapping("/search")
    public String searchProduct(
            @RequestParam(defaultValue = "General") String category,
            @RequestParam(defaultValue = "1000") int maxPrice) {

        return "Displaying " + category + " under " + maxPrice;
    }
}
```

Get B

1. 1) Centralized Exception Handling: Calculator Division API.

```
Save in Demo project:

Main File:
demo/src/main/java/com/example/demo/DemoApplication.java

Controller File:
demo/src/main/java/com/example/demo/DivisionController.java

Exception Handler File:
demo/src/main/java/com/example/demo/GlobalErrorHandler.java

Program:

-----------------------------------------------------------------------------------------------
// DivisionController.java
package com.example.demo;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class DivisionController {

    @GetMapping("/divide")
    public String divide(@RequestParam int a, @RequestParam int b) {
        int result = a / b;
        return "Division = " + result;
    }
}
```

```
// GlobalErrorHandler.java
package com.example.demo;

import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.ResponseBody;

@ControllerAdvice
public class GlobalErrorHandler {

    @ExceptionHandler(ArithmeticException.class)
    @ResponseBody
    public String handleArithmeticException(ArithmeticException ex) {
        return "Error: Division by zero is not allowed.";
    }
}
```

2. 2) API Documentation with Swagger: Student Management API.

```
Save in Demo project:

Main File:
demo/src/main/java/com/example/demo/DemoApplication.java

Controller File:
demo/src/main/java/com/example/demo/StudentApiController.java

pom.xml File:
demo/pom.xml

Program:

-----------------------------------------------------------------------------------------------------
// DemoApplication.java
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import io.swagger.v3.oas.annotations.OpenAPIDefinition;
import io.swagger.v3.oas.annotations.info.Info;

@SpringBootApplication
@OpenAPIDefinition(
    info = @Info(
        title = "College Portal API",
        version = "1.0",
        description = "Student Management API"
    )
)
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}


-----------------------------------------------------------------------------------------------------

// StudentApiController.java
package com.example.demo;

import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class StudentApiController {

    @PostMapping("/registerStudent")
    public String registerStudent(@RequestBody String name) {
        return "Student " + name + " Registered";
    }
}
```

```
// pom.xml dependency
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.8.13</version>
</dependency>

Swagger URL:
http://localhost:8080/swagger-ui/index.html
```

Mass . 5

Common Project Creation Process

```
1. Eclipse open kara
2. File -> New -> Spring Starter Project
3. Project Name dya
4. Type: Maven
5. Packaging: Jar
6. Java Version: 17
7. Group: com.example
8. Artifact: project name
9. Dependencies select kara:
   - Spring Web
   - Spring Data JPA
   - PostgreSQL Driver
   - Thymeleaf (Set A sathi)
10. Finish
```

Common pom.xml Dependency

```
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>

    <dependency>
        <groupId>org.postgresql</groupId>
        <artifactId>postgresql</artifactId>
        <scope>runtime</scope>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-thymeleaf</artifactId>
    </dependency>
</dependencies>
```

Database Query

```
create database studentdb;
```

Common application.properties Path

```
src/main/resources/application.properties
```

Common application.properties

```
spring.datasource.url=jdbc:postgresql://localhost:5432/studentdb
spring.datasource.username=postgres
spring.datasource.password=postgres

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

spring.thymeleaf.cache=false
server.port=8080
```

Common Folder Structure

```
project-name
 └── src/main/java/com/example/projectname
      ├── Application.java
      ├── controller/
      ├── entity/
      ├── repository/
 └── src/main/resources
      ├── templates/
      └── application.properties
 └── pom.xml
```

Get A

1) Student form accept kara ani PostgreSQL madhe save kara

```
Project Name: assignment5seta
------------------------------
Main Class:
src/main/java/com/example/assignment5seta/Assignment5SetAApplication.java

Entity Class:
src/main/java/com/example/assignment5seta/entity/Student.java

Repository Interface:
src/main/java/com/example/assignment5seta/repository/StudentRepository.java

REST Controller:
src/main/java/com/example/assignment5seta/controller/StudentController.java
```

HTML Form:
src/main/resources/templates/studentform.html

Properties File:
src/main/resources/application.properties

```
package com.example.assignment5seta;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class Assignment5SetAApplication {
    public static void main(String[] args) {
        SpringApplication.run(Assignment5SetAApplication.class, args);
    }
}
```

```
package com.example.assignment5seta.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;

@Entity
public class Student {

    @Id
    private int id;
    private String name;
    private String email;

    public Student() {
    }

    public Student(int id, String name, String email) {
        this.id = id;
        this.name = name;
        this.email = email;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }
```

```
    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }
}
```

```
package com.example.assignment5seta.repository;

import java.util.List;
import org.springframework.data.domain.Sort;
import org.springframework.data.jpa.repository.JpaRepository;
import com.example.assignment5seta.entity.Student;

public interface StudentRepository extends JpaRepository<Student, Integer> {
    List<Student> findByNameContaining(String name);
    List<Student> findAll(Sort sort);
}
```

```
package com.example.assignment5seta.controller;

import java.util.List;
import java.util.Optional;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.*;

import com.example.assignment5seta.entity.Student;
import com.example.assignment5seta.repository.StudentRepository;

@Controller
public class StudentController {

    @Autowired
    private StudentRepository repository;
```

```
    @GetMapping("/form")
    public String showForm() {
        return "studentform";
    }

    @PostMapping("/saveStudent")
    @ResponseBody
    public String saveStudent(@RequestParam int id,
                              @RequestParam String name,
                              @RequestParam String email) {
        Student s = new Student(id, name, email);
        repository.save(s);
        return "Student Saved Successfully";
    }

    @GetMapping("/students")
    @ResponseBody
    public List<Student> getAllStudents() {
        return repository.findAll();
    }

    @GetMapping("/students/search")
    @ResponseBody
    public List<Student> searchByName(@RequestParam String name) {
        return repository.findByNameContaining(name);
    }

    @PutMapping("/students/{id}")
    @ResponseBody
    public String updateStudentEmail(@PathVariable int id, @RequestParam String
email) {
        Optional<Student> op = repository.findById(id);

        if (op.isPresent()) {
            Student s = op.get();
            s.setEmail(email);
            repository.save(s);
            return "Student Email Updated";
        } else {
            return "Student Not Found";
        }
    }

    @DeleteMapping("/students/{id}")
    @ResponseBody
    public String deleteStudent(@PathVariable int id) {
        Optional<Student> op = repository.findById(id);

        if (op.isPresent()) {
            repository.deleteById(id);
```

```
            return "Student Deleted";
        } else {
            return "Student Not Found";
        }
    }

    @GetMapping("/students/sorted")
    @ResponseBody
    public List<Student> getSortedStudents() {
        return repository.findAll(Sort.by("name").ascending());
    }
}
```

```
<!DOCTYPE html>
<html>
<body>
    <h2>Student Form</h2>
    <form action="/saveStudent" method="post">
        ID: <input type="text" name="id"><br><br>
        Name: <input type="text" name="name"><br><br>
        Email: <input type="text" name="email"><br><br>
        <input type="submit" value="Save">
        <input type="reset" value="Clear">
    </form>
</body>
</html>
```

-

Get A - API Testing

```
Browser:
http://localhost:8080/form

Postman:
GET    http://localhost:8080/students
GET    http://localhost:8080/students/search?name=Am
PUT    http://localhost:8080/students/1?email=new@gmail.com
DELETE http://localhost:8080/students/1
GET    http://localhost:8080/students/sorted
```

Get B

1) Product Catalog + Inventory Update

```
Project Name: assignment5setb

Main Class:
src/main/java/com/example/assignment5setb/Assignment5SetBApplication.java

Product Entity:
src/main/java/com/example/assignment5setb/entity/Product.java

Product Repository:
src/main/java/com/example/assignment5setb/repository/ProductRepository.java

Product Controller:
src/main/java/com/example/assignment5setb/controller/ProductController.java

Properties File:
src/main/resources/application.properties
```

```
package com.example.assignment5setb;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class Assignment5SetBApplication {
    public static void main(String[] args) {
        SpringApplication.run(Assignment5SetBApplication.class, args);
    }
}
```

```
package com.example.assignment5setb.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;

@Entity
public class Product {

    @Id
    private int id;
    private String name;
    private double price;

    public Product() {
    }
```

```
    public Product(int id, String name, double price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public double getPrice() {
        return price;
    }

    public void setPrice(double price) {
        this.price = price;
    }
}
```

```
package com.example.assignment5setb.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.example.assignment5setb.entity.Product;

public interface ProductRepository extends JpaRepository<Product, Integer> {
}
```

```
package com.example.assignment5setb.controller;

import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import com.example.assignment5setb.entity.Product;
import com.example.assignment5setb.repository.ProductRepository;
```

```
@RestController
public class ProductController {

    @Autowired
    private ProductRepository repository;

    @PostMapping("/products")
    public String addProduct(@RequestBody Product product) {
        repository.save(product);
        return "Product Added Successfully";
    }

    @GetMapping("/products")
    public List<Product> getAllProducts() {
        return repository.findAll();
    }

    @PutMapping("/products/{id}")
    public String updateProductPrice(@PathVariable int id, @RequestParam double
price) {
        Product p = repository.findById(id).orElse(null);

        if (p == null) {
            return "Product Not Found";
        }

        p.setPrice(price);
        repository.save(p);
        return "Product Price Updated";
    }
}
```

---

2) Employee Search

Save in same project: assignment5setb

Employee Entity:
src/main/java/com/example/assignment5setb/entity/Employee.java

Employee Repository:
src/main/java/com/example/assignment5setb/repository/EmployeeRepository.java

Employee Controller:
src/main/java/com/example/assignment5setb/controller/EmployeeController.java

```java
package com.example.assignment5setb.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;

@Entity
public class Employee {

    @Id
    private int id;
    private String name;
    private String department;

    public Employee() {
    }

    public Employee(int id, String name, String department) {
        this.id = id;
        this.name = name;
        this.department = department;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getDepartment() {
        return department;
    }

    public void setDepartment(String department) {
        this.department = department;
    }
}
```

```
package com.example.assignment5setb.repository;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import com.example.assignment5setb.entity.Employee;

public interface EmployeeRepository extends JpaRepository<Employee, Integer> {
    List<Employee> findByDepartment(String dept);
}
```

```
package com.example.assignment5setb.controller;

import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import com.example.assignment5setb.entity.Employee;
import com.example.assignment5setb.repository.EmployeeRepository;

@RestController
public class EmployeeController {

    @Autowired
    private EmployeeRepository repository;

    @PostMapping("/employees")
    public String addEmployee(@RequestBody Employee employee) {
        repository.save(employee);
        return "Employee Added Successfully";
    }

    @GetMapping("/employees/search")
    public List<Employee> searchByDepartment(@RequestParam String dept) {
        return repository.findByDepartment(dept);
    }
}
```

Get B API Testing

```
POST http://localhost:8080/products
Body -> raw -> JSON
{
  "id": 1,
  "name": "Laptop",
  "price": 55000
}
```

GET http://localhost:8080/product

s

```
PUT http://localhost:8080/products/1?price=60000

POST http://localhost:8080/employees
Body -> raw -> JSON
{
  "id": 1,
  "name": "Amit",
  "department": "IT"
}

GET http://localhost:8080/employees/search?dept=IT
```

How to Run in openSUSE Linux with Eclipse

```
1. PostgreSQL open kara
2. create database studentdb; run kara
3. Spring Starter Project create kara
4. Dependencies add kara
5. application.properties set kara
6. Files exact path nusar save kara
7. Main class run kara
8. Postman madhye API test kara
```

Spring boot application add, mul, sub, divide

```
package com.example.assignment3c;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;

@Controller
public class CalculatorController {

    @GetMapping("/")
    public String showForm() {
        return "calculator";
    }

    @PostMapping("/calculate")
    public String calculate(@RequestParam int a,
                            @RequestParam int b,
                            @RequestParam String operation,
                            Model model) {

        String result;

        switch (operation) {
            case "add":
                result = "Addition = " + (a + b);
```

```
            break;
        case "sub":
            result = "Subtraction = " + (a - b);
            break;
        case "mul":
            result = "Multiplication = " + (a * b);
            break;
        case "div":
            if (b == 0) {
                result = "Division by zero not allowed";
            } else {
                result = "Division = " + (a / b);
            }
            break;
        default:
            result = "Invalid operation";
    }

    model.addAttribute("result", result);
    return "calculator";
  }
}
```

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <title>Calculator</title>
  <meta charset="UTF-8">
```

```
</head>
<body>
  <h2>Simple Calculator</h2>

  <form action="/calculate" method="post">
    Enter First Number:
    <input type="number" name="a" required><br><br>

    Enter Second Number:
    <input type="number" name="b" required><br><br>

    Select Operation:
    <select name="operation">
      <option value="add">Addition</option>
      <option value="sub">Subtraction</option>
      <option value="mul">Multiplication</option>
      <option value="div">Division</option>
    </select><br><br>

    <input type="submit" value="Calculate">
    <input type="reset" value="Reset">
  </form>

  <h3 th:text="${result}"></h3>
</body>
</html>
```